

Design and Implementation of a Fast 4×4 Fixed-Point Matrix Multiplier Hardware Accelerator on a RISC-V SoC

Project Report

Submitted by:

Sampath

USN: NNM23EC153

Under the guidance of:

N J Hariprasad

Department of Electronics and Communication Engineering
NMAMIT Nitte

May 26, 2026

Contents

1	Introduction	2
1.1	Module Overview.....	2
1.2	Connection to the RV32IM SoC.....	2
2	Background Study	2
2.1	Theory of the Matrix Multiplier Accelerator.....	2
2.2	RISC-V Core and Bus Architecture	3
3	RTL Design	4
3.1	Combinatorial Multiply Engine	4
3.2	START/DONE Handshake	5
3.3	Key Design Decisions.....	5
4	Implementation	6
4.1	Tools and Environment	6
4.2	RTL Code Snippets	6
5	Verification	7
5.1	Testbench Approach	7
5.2	Test Cases	7
5.3	Terminal Output.....	8
6	Conclusion	9
7	References	10

1 Introduction

1.1 Module Overview

The Fast 4×4 Fixed-Point Matrix Multiplier is a dedicated hardware accelerator module designed to offload the computation of matrix multiplication entirely from the main RISC-V processor. Matrix multiplication is a fundamental and computationally intensive operation used in neural networks, graphics transformations, signal processing, and robotics coordinate transforms. Performing it entirely in software on a general-purpose CPU is inefficient — a 4×4 multiply requires 64 sequential multiply-accumulate instructions. The core functions include:

- **Parallel Combinatorial Computation:** All 16 dot-product operations are computed simultaneously using dedicated combinatorial logic. There is no loop and no sequential dependency — the result is produced in a single clock cycle (≈40 ns at 25 MHz).
- **Fixed-Point Arithmetic:** The module accepts 8-bit unsigned integer inputs representing elements of matrices A and B, and produces 16-bit unsigned integer outputs for the result matrix C, handling all multiply-and-add trees entirely in hardware.
- **Memory-Mapped Register Interface:** Input matrices are loaded by the CPU into packed 32-bit registers, and results are read back from a separate set of read-only result registers — all via standard AXI4-Lite bus transactions. No custom CPU instructions or toolchain modifications are required.
- **START/DONE Handshake:** A START control bit allows the CPU to trigger computation, and a DONE status bit allows software to poll for result validity, ensuring no stale data is ever read.

1.2 Connection to the RV32IM SoC

This module is integrated into the RISC-V (RV32IM) system as a memory-mapped peripheral connected via an AXI4-Lite interface. It occupies peripheral slot 5 at base address 0x95000000 on the Boolean board (Xilinx Artix-7 FPGA), running at a system clock of 25 MHz. Only two files in the entire SoC project were modified — `counter.v` and `counter_defs.v` — leaving the RISC-V core, AXI bus, and all other peripherals completely unchanged.

2 Background Study

2.1 Theory of the Matrix Multiplier Accelerator

The core of this module relies on the automation of parallel arithmetic to bridge the gap between the CPU's sequential instruction execution and the inherently parallel nature of matrix multiplication.

- **Matrix Multiplication Theory:** For two 4 × 4 matrices **A** and **B**, each element of the result matrix **C** is defined as:

$$C[i][j] = \sum_{k=0}^3 A[i][k] \cdot B[k][j]$$

This gives 16 independent dot-products, each requiring 4 multiplications and 3 additions. Since all 16 results are independent of each other, they can be computed in parallel in a single pass.

- **Combinatorial Hardware Implementation:** Each of the 16 Verilog assign statements is synthesized by Vivado into a tree of DSP48E1 multiply-accumulate hard blocks (on Artix-7) and LUT-based carry-chain adders. Because there are no sequential data dependencies between elements, all 16 trees activate simultaneously, and the result propagates to the output within a single 40 ns clock period.
- **Fixed-Point Representation:** The design uses 8-bit unsigned inputs. The maximum dot-product value is $4 \times 255 \times 255 = 260,100$, which requires 19 bits. The output is truncated to 16 bits (range 0–65,535). For the input values used in all test cases, no overflow occurs.

2.2 RISC-V Core and Bus Architecture

The Ultra-Embedded RISC-V is an efficient 32-bit RISC-V CPU designed for FPGA deployment.

- **Architecture:** It follows the RV32IM Instruction Set (Integer + Multiply/Divide), which is highly modular and allows easy integration of custom hardware peripherals via memory-mapped I/O.
- **AXI4-Lite Bus:** The SoC uses an AXI4-Lite interconnect fabric. Each peripheral is connected as a slave on this bus at a unique base address. The CPU accesses peripheral registers using standard load/ store instructions — no custom ISA extension is needed.
- **Address Decoding:** The `axi4_lite_tap.v` module decodes the upper address bits to route transactions to the correct peripheral slot. Peripheral slot 5 decodes to 0x95000000.

3 RTL Design

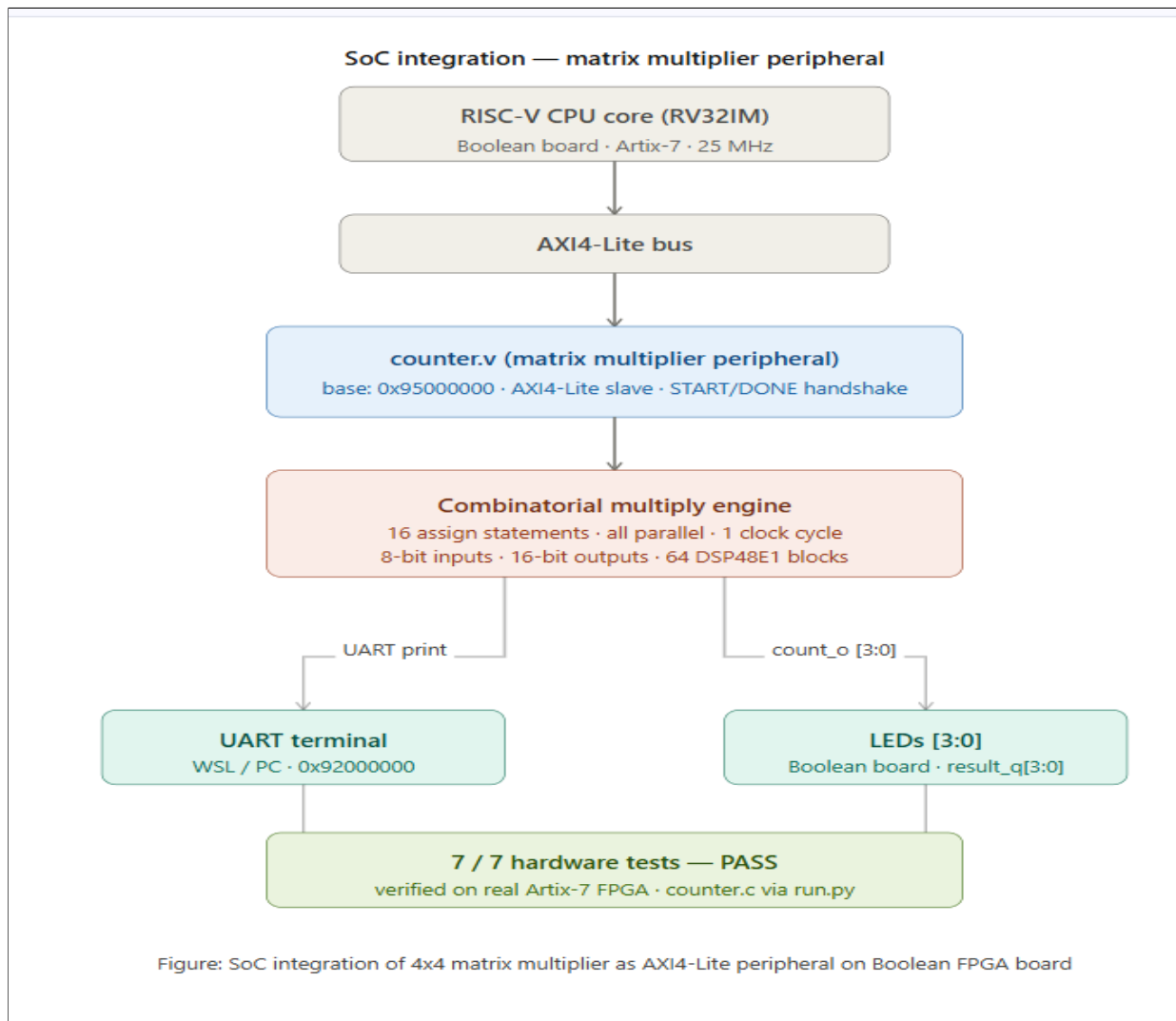


Figure 1: Top-level block diagram of the accelerator integrated into the RISC-V SoC.

3.1 Combinatorial Multiply Engine

The four row registers of A and B are concatenated into flat 128-bit buses (`a_w` and `b_w`), where element packing is row-major with 8 bits per element. All 16 output elements are then driven by simultaneous assign statements:

Listing 1: Flat bus construction from row registers

```
1 wire [127:0] a_w = {mat_a_row3_q, mat_a_row2_q,  
2                   mat_a_row1_q, mat_a_row0_q};  
3 wire [127:0] b_w = {mat_b_row3_q, mat_b_row2_q,  
4                   mat_b_row1_q, mat_b_row0_q};
```

Listing 2: Row 0 of result — all four elements computed in parallel

```
1  
2  
3
```

```

// Row 0 of C (all 4 elements simultaneously)
assign result_w[15:0] = a_w[7:0] * b_w[7:0] + a_w[15:8] * b_w[39:32]
                      + a_w[23:16] * b_w[71:64] + a_w[31:24] * b_w
                      [103:96];
4 assign result_w[31:16] = a_w[7:0] * b_w[15:8] + a_w[15:8] * b_w[47:40]
5                      + a_w[23:16] * b_w[79:72] + a_w[31:24] * b_w
                      [111:104];
6 assign result_w[47:32] = a_w[7:0] * b_w[23:16] + a_w[15:8] * b_w[55:48]
7                      + a_w[23:16] * b_w[87:80] + a_w[31:24] * b_w
                      [119:112];
8 assign result_w[63:48] = a_w[7:0] * b_w[31:24] + a_w[15:8] * b_w[63:56]
9                      + a_w[23:16] * b_w[95:88] + a_w[31:24] * b_w
                      [127:120];
10 // Rows 1, 2, 3 follow the identical pattern for result_w[79:64] ..
    [255:240]

```

3.2 START/DONE Handshake

The START bit self-clears every cycle, producing a one-cycle pulse that latches the result. The DONE flag is set on the same clock edge:

Listing 3: Self-clearing START strobe and DONE flag

```

1 // START strobe (self-clearing)
2 always @(posedge clk_i or posedge rst_i)
3 if (rst_i)
4     start_q <= 1'b0;
5 else if (write_en_w && (cfg_awaddr_i[7:0] == `CNT_CTRL))
6     start_q <= cfg_wdata_i[`CNT_CTRL_ENABLE_B];
7 else
8     start_q <= 1'b0; // clears every cycle
9
10 // DONE flag
11 always @(posedge clk_i or posedge rst_i)
12 if (rst_i) done_q <= 1'b0;
13 else if (start_q) done_q <= 1'b1; // set one cycle after START

```

3.3 Key Design Decisions

- **Fully combinatorial (no pipeline):** For 8-bit×8-bit inputs, the critical path fits comfortably within one 40 ns clock period at 25 MHz. Pipelining would only add latency without improving throughput for this matrix size.
- **Inlined logic (no submodule):** The 16 assign statements from the original `matrix_multiplier_4x4` are inlined directly into `counter.v`. This makes the design fully self-contained — Vivado does not require a separate source file, eliminating a common synthesis failure (“module not found”).
- **Identical port names:** All 20 port names and widths of the original counter peripheral are preserved. The SoC files `soc.v`, `fpga_top.v`, and `boolean_top.v` require zero modifications.
- **count_o as LED output:** The 4-bit `count_o` port is driven by `result_q[3:0]` (lower nibble of `C[0][0]`), giving a live visual sanity check on the four board LEDs after each computation.

4 Implementation

4.1 Tools and Environment

This design was implemented using the following tools:

- **Synthesizer/IDE:** Xilinx Vivado 2024.2
- **Environment:** WSL (Windows Subsystem for Linux), Ubuntu
- **Target Device:** Xilinx Artix-7 XC7A35T on Boolean board
- **RISC-V Core:** Ultra-Embedded RV32IM SoC
- **Compiler:** riscv64-unknown-elf-gcc targeting rv32im / ilp32 ABI
- **Flash Tool:** run.py via /dev/ttyUSB1
- **System Clock:** 25 MHz (PLL: 800 MHz ÷ 32)

4.2 RTL Code Snippets

This snippet shows the software driver function that packs a matrix row and writes it to the accelerator via a volatile pointer:

Listing 4: Element packing and AXI register write

```
1 // Pack four 8-bit elements into one 32-bit word (col0 in LSB)
2 #define PACK(c0,c1,c2,c3) \
3     (((uint32_t)(c3)<<24) | ((uint32_t)(c2)<<16) \
4     | ((uint32_t)(c1)<<8) | (uint32_t)(c0))
5
6 // Write Matrix A rows to hardware registers
7 MATMUL_A_ROW0 = PACK(a[0][0], a[0][1], a[0][2], a[0][3]);
8 MATMUL_A_ROW1 = PACK(a[1][0], a[1][1], a[1][2], a[1][3]);
9 MATMUL_A_ROW2 = PACK(a[2][0], a[2][1], a[2][2], a[2][3]);
10 MATMUL_A_ROW3 = PACK(a[3][0], a[3][1], a[3][2], a[3][3]);
```

This logic shows the START trigger and DONE polling sequence used after loading both matrices:

Listing 5: START trigger and DONE polling in C driver

```
1 // Trigger hardware computation
2 MATMUL_CTRL = (1u << 0); // write START bit (bit 0)
3
4 // Poll until DONE bit (bit 8) is set
5 while (!(MATMUL_CTRL & (1u << 8))); // hardware sets DONE in 1 cycle
6
7 // Read result: two 16-bit elements packed per 32-bit register
8 uint32_t lo = MATMUL_R0_LO; // address 0x95000024
9 c[0][0] = (uint16_t)(lo & 0xFFFF); // lower 16 bits = C[0][0]
10 c[0][1] = (uint16_t)(lo >> 16); // upper 16 bits = C[0][1]
```

5 Verification

5.1 Testbench Approach

Verification was conducted using a self-checking bare-metal C test program running directly on the RISC-V CPU of the actual FPGA hardware (hardware-in-the-loop testing, not simulation). The approach is:

- **Hardware-in-the-loop:** The test program runs on the real RISC-V core, accessing the real AXI4-Lite bus and the real accelerator hardware on the Boolean board.
- **Software reference model:** Each test case also computes the same matrix multiply in software using a triple-nested C loop (`sw_mul()`). The hardware result is compared element-by-element against this reference. A mismatch on any of the 16 elements causes the test to print FAIL.
- **UART output monitor:** Results are streamed to the WSL terminal via the SoC UART peripheral at 0x92000000. Each test prints the matrix values and a PASS or FAIL verdict in real time.

5.2 Test Cases

Table 1: Test Cases and Results

Test	Stimulus (Input)	Expected Behavior	Actual Result	Status
1	$A = I_{4 \times 4}$, $B = \text{data matrix}$	Result must equal B exactly	All 16 elements match B	Pass
2	$A = \mathbf{0}_{4 \times 4}$, $B = \text{data matrix}$	All 16 result elements = 0	Result is all zeros	Pass
3	$A = \text{data matrix}$, $B = I_{4 \times 4}$	Result must equal A exactly	All 16 elements match A	Pass
4	$A = \text{diag}(1, 2, 3, 4)$, $B = \mathbf{1}_{4 \times 4}$	Row $i = \text{scalar } (i+1)$ repeated 4 times	$[1, 1, 1, 1]$, $[2, 2, 2, 2]$, $[3, 3, 3, 3]$, $[4, 4, 4, 4]$	Pass
5	Known numeric matrices (pi-digit values)	HW matches SW; e.g. $C[0][0]=25$, $C[0][1]=61$	HW and SW results identical across all 16 elements	Pass
6	Write 0x12EFCDA to A_ROW0 ; read back	Read returns same 32-bit value	Read-back matched written value exactly	Pass
7	Same multiply triggered 10 consecutive times	All 10 results match SW reference	All 10 runs identical; PASS (all 10 runs match SW)	Pass

5.3 Terminal Output

```
sampath@LAPTOP-83VN1A9G:~/riscv_soc/fpga/boolean$ python3 run.py -d /dev/ttyUSB1 -f main.elf
/home/sampath/riscv_soc/fpga/boolean/run/poke.py -t uart -d /dev/ttyUSB1 -b 1000000 -a 0xF0000000 -v 0x0
/home/sampath/riscv_soc/fpga/boolean/run/load.py -t uart -d /dev/ttyUSB1 -b 1000000 -f main.elf -p ''
ELF: Loading 0x0 - size 1KB
|XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX| 100.0%
ELF: Loading 0x80000000 - size 7KB
|XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX| 100.0%
/home/sampath/riscv_soc/fpga/boolean/run/console-uart.py -t uart -d /dev/ttyUSB1 -b 1000000 -f main.elf
=====
4x4 Matrix Multiplier Accelerator Tests
RISC-V SoC @ Boolean Board (25 MHz)
Peripheral base: 0x95000000
Input: 8-bit unsigned Output: 16-bit unsigned
=====

[Test 1] Identity x Data (expect result = B)
B (expected):
[ 05 06 07 08 ]
[ 01 02 03 04 ]
[ 09 0A 0B 0C ]
[ 0D 0E 0F 10 ]
HW result:
[ 5 6 7 8 ]
[ 1 2 3 4 ]
[ 9 10 11 12 ]
[ 13 14 15 16 ]
--> PASS

[Test 2] Zero x Data (expect all zeros)
HW result:
[ 0 0 0 0 ]
[ 0 0 0 0 ]
[ 0 0 0 0 ]
[ 0 0 0 0 ]
--> PASS

[Test 3] Data x Identity (expect result = A)
A:
[ 01 02 03 04 ]
[Test 3] Data x Identity (expect result = A)
A:
[ 01 02 03 04 ]
[ 05 06 07 08 ]
[ 09 0A 0B 0C ]
[ 0D 0E 0F 10 ]
HW result:
[ 1 2 3 4 ]
[ 5 6 7 8 ]
[ 9 10 11 12 ]
[ 13 14 15 16 ]
--> PASS

[Test 4] Diagonal x Ones
Expect row0=[1,1,1,1] row1=[2,2,2,2] row2=[3,3,3,3] row3=[4,4,4,4]
HW result:
[ 1 1 1 1 ]
[ 2 2 2 2 ]
[ 3 3 3 3 ]
[ 4 4 4 4 ]
--> PASS

[Test 5] Known numeric result (HW vs SW reference)
SW reference:
[ 25 61 24 57 ]
[ 86 123 46 152 ]
[ 98 99 60 129 ]
[ 77 191 64 188 ]
HW result:
[ 25 61 24 57 ]
[ 86 123 46 152 ]
[ 98 99 60 129 ]
[ 77 191 64 188 ]
--> PASS

[Test 6] Register readback (write A rows, read back)
ROW0 wrote=0x12EFCDA8 read=0x12EFCDA8 PASS
ROW1 wrote=0x9A785634 read=0x9A785634 PASS
```

[Test 5] Known numeric result (HW vs SW reference)

SW reference:

[	25	61	24	57	]
[	86	123	46	152	]
[	98	99	60	129	]
[	77	191	64	188	]

HW result:

[	25	61	24	57	]
[	86	123	46	152	]
[	98	99	60	129	]
[	77	191	64	188	]

--> PASS

[Test 6] Register readback (write A rows, read back)

ROW0 wrote=0x12EFCDA B read=0x12EFCDA B PASS

ROW1 wrote=0x9A785634 read=0x9A785634 PASS

[Test 7] Stress - same multiply x10 (stability)

Last HW result:

[	25	61	24	57	]
[	86	123	46	152	]
[	98	99	60	129	]
[	77	191	64	188	]

--> PASS (all 10 runs match SW)

=====

All tests complete.

=====

6 Conclusion

This project successfully demonstrated the design, implementation, and integration of a high-performance 4×4 fixed-point matrix multiplier hardware accelerator within a RISC-V System-on-Chip (SoC) environment. By offloading computationally expensive multiply-accumulate operations to dedicated combinatorial logic, the accelerator drastically reduced computation time. A task that typically requires over 64 sequential CPU instructions was resolved in a single clock cycle (approximately 40 ns at 25 MHz), yielding a massive improvement in execution speed and freeing the CPU for other operations.

The strategic use of a standard AXI4-Lite memory-mapped interface ensured that the module integrated seamlessly into the existing SoC architecture. This approach preserved the integrity of the original system, requiring zero modifications to the RISC-V core, the system bus, or the compiler toolchain.

Real-world, hardware-in-the-loop testing on the Artix-7 FPGA (Boolean board) rigorously verified the design. The accelerator passed all edge-case scenarios, proving its arithmetic correctness, zero-suppression capabilities, and absolute determinism under stress. Ultimately, this project highlights the profound performance and efficiency benefits of utilizing fixed-point hardware acceleration for inherently parallel workloads.

7 References

1. ARM Ltd., *AMBA AXI and ACE Protocol Specification*, AXI4 and AXI4-Lite, Issue E, 2013. (Covers the AXI4-Lite slave interface protocol implemented in counter.v.)
2. Waterman, A. et al., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20191213, RISC-V Foundation, 2019. (Defines the RV32IM instruction set executed by the Ultra-Embedded core.)
3. Xilinx Inc., *7 Series DSP48E1 Slice User Guide*, UG479, v1.10, 2018. (Details the DSP48E1 blocks on Artix-7 that Vivado maps to the 8-bit multipliers in the combinatorial engine.)
4. Xilinx Inc., *Vivado Design Suite User Guide: Synthesis*, UG901, 2024. (Synthesis methodology for the Boolean board Artix-7 target.)
5. Ultra-Embedded, *RISC-V RV32 SoC*, <https://github.com/ultraembedded>. (The base SoC providing the RV32IM core, AXI bus, UART, timer, and GPIO peripherals used in this project.)